

Rapport d'activité – Rendu 3

Partie 1 — Dynamiques du jeu

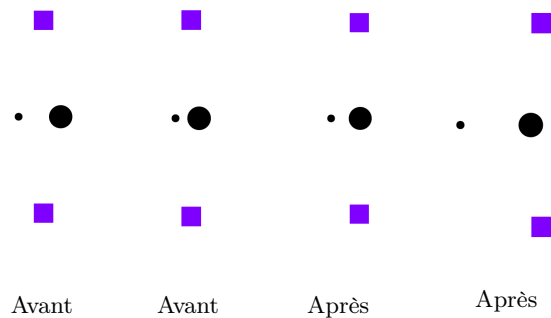
1.1 Changements structurels par rapport au Rendu 2 (optionnel)

- **Ball** : ajout des méthodes `translate()` et `setDelta()` pour rendre la classe mutable.
- **Paddle** : ajout de l'attribut `last_delta` et de la méthode `getDelta()` pour la collision balle-raquette.
- **Brick** : ajout de la méthode virtuelle pure `hit()` dans la hiérarchie de classes.
- **Game** : réécriture complète de `update()` avec les trois phases de la dynamique.

1.2 Collision de deux balles de rayons différents

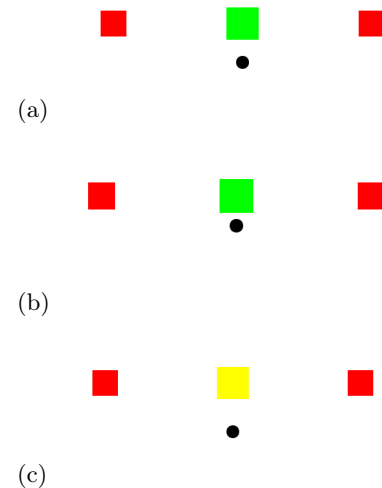
La balle A (petite, rayon r_a) se déplace et entre en contact avec la balle B (grande, rayon r_b). La collision est détectée dès que la distance entre leurs centres est inférieure à $r_a + r_b + \epsilon$.

Lors de l'impact, les deux balles rebondissent : une impulsion est appliquée le long de l'axe reliant leurs centres, pondérée par le rapport des rayons. Les composantes tangentielles restent inchangées et les normes des deltas sont bornées à `delta_norm_max`.



1.3 Destruction de chaque type de brique

RainbowBrick : à chaque impact, la brique perd 1 `hit_point` et sa couleur change. Elle est détruite lorsque ses points de vie atteignent zéro.

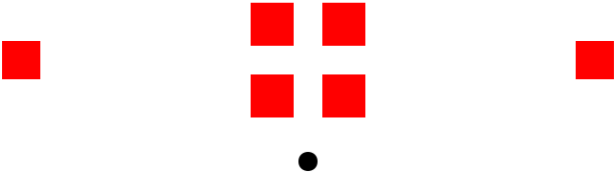


BallBrick : détruite dès le premier impact. Une nouvelle balle est générée au centre de la brique, héritant du delta incident de la balle frappante.

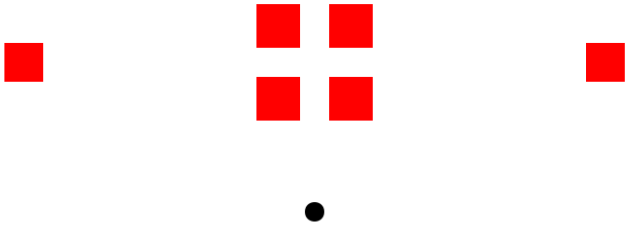




SplitBrick : détruite au premier impact. (a)
Elle spawnne 4 briques filles de taille $(size - split_brick_gap)/2$, uniquement si cette
taille $\geq brick_size_min$.



(b)



(c)

Partie 2 — Travail en groupe et bilan

2.1 Activité individuelle

- **Liam Van Camp** : *[Décrivez votre contribution — modules, fonctions, tests, etc.]*
- **Victor Eder** : *[Décrivez sa contribution — modules, fonctions, tests, etc.]*

2.2 Méthodologie

Organisation et outils : *[Décrivez comment vous vous êtes organisés — Git (branches, commits), VSCode Live Share, réunions, etc.]*

Ordre de développement et tests : *[Dans quel ordre avez-vous développé les modules? Ex : tools → ball/brick/paddle → game : :load → game : :update. Quelles méthodes de test pour chaque partie (fichiers .txt manuels, débogage visuel, etc.) ?]*

Travail simultané vs indépendant : *[Quelle proportion du code a été écrite côte à côte (pair programming) vs en parallèle de façon indépendante ? Avec le recul, changeriez-vous cette proportion, et pourquoi ?]*

2.3 Bug le plus fréquent

[Décrivez le bug qui revenait le plus souvent — ex : balle traversant les briques (effet tunnel), boucle infinie dans update(), mauvaise direction nominale, etc. Quelle était la cause, comment l'avez-vous détecté et résolu ?]

2.4 Usage d'un outil d'IA

[Avez-vous utilisé un outil d'IA (Claude, ChatGPT, Copilot, etc.) ? Si oui, pour quelles tâches (compréhension des formules de collision, débogage, structuration du rapport, etc.) ? Gain ou perte de temps — quantifiez approximativement (ex : “gain d'environ 1h sur la compréhension de la formule d'impulsion balle-balle”).]

2.5 Conclusion

[Auto-évaluation courte sur deux axes :]

Code : *[Robustesse (gestion des cas limites, boucles infinies évitées via nb_bounce_max, etc.) et performance (complexité des boucles dans update(), etc.)]*

Environnement de travail : *[Points forts et points faibles de l'environnement fourni (GTKmm, structure imposée, fichiers de test, etc.) et améliorations que vous suggéreriez.]*